Занятие 12: Интеграционные проверки и диагностика

Тестирование с реальной БД, логирование, request ID

Слайд 1: Почему изолированные тесты могут проходить?

Пример проблемы:

```
# Unit test - проходит
def test_create_user():
    mock_repo = Mock()
    mock_repo.save.return_value = User(id=1, name="John")
    service = UserService(mock_repo)
    user = service.create_user("John", "john@example.com")
    assert user.id == 1  # ✓ PASS
```

Но в реальности:

- База данных недоступна
- Схема БД не соответствует модели
- Ограничения UNIQUE нарушены
- Транзакция не коммитится

Результат: Тест проходит, но приложение не работает.

Слайд 2: Интеграционные тесты

Что проверяют интеграционные тесты:

- Взаимодействие компонентов
- Интеграцию с БД
- Интеграцию с внешними API
- Конфигурацию окружения

Пример:

```
def test_create_user_integration():
    # Реальная БД
    conn = get_test_db_connection()
    repo = UserRepository(conn)
    service = UserService(repo)

    user = service.create_user("John", "john@example.com")

    # Проверка в БД
    saved_user = repo.find_by_id(user.id)
    assert saved_user is not None
    assert saved_user.email == "john@example.com"
```

Слайд 3: Тестовое окружение

Отдельное тестовое окружение:

Почему нужно:

- Изоляция от production данных
- Возможность уничтожать данные
- Быстрее (меньше данных)
- Стабильные тестовые данные

Пример конфигурации:

```
# test_config.py
TEST_DATABASE_URL = "postgresql://test:test@localhost:5432/test_db"
TEST_REDIS_URL = "redis://localhost:6379/1"
TEST_API_URL = "http://localhost:8001"
```

Docker Compose для тестов:

```
services:
  test-db:
    image: postgres:15
    environment:
      POSTGRES_DB: test_db
```

Слайд 4: Подготовка тестовых данных

Fixture для подготовки данных:

```python
@pytest.fixture
def test_db():
    """Setup test database with clean state"""
    conn = psycopg2.connect(TEST_DATABASE_URL)
    cursor = conn.cursor()

    # Clean tables
    cursor.execute("TRUNCATE TABLE users CASCADE")
    conn.commit()

    # Insert test data
    cursor.execute("INSERT INTO users (name, email) VALUES (%s, %s)", ("Test User", "test@example.com"))
    conn.commit()

    yield conn

    # Cleanup
    cursor.execute("TRUNCATE TABLE users CASCADE")
    conn.commit()
    cursor.close()
    conn.close()
```

Использование:

```python
def test_get_user(test_db):
    repo = UserRepository(test_db)
    user = repo.find_by_email("test@example.com")
    assert user is not None
```

Слайд 5: Независимость тестов

Проблема зависимости от порядка:

```
# Test 1
def test_create_user():
    user = create_user("John")
    assert user.id == 1

# Test 2 - зависит от Test 1!
def test_get_user():
    user = get_user(1)  # Ошибка если Test 1 не запущен!
```

Решение - изоляция:

```
@pytest.fixture(autouse=True)
def clean_db():
    """Clean database before each test"""
    conn = get_test_db()
    cursor = conn.cursor()
    cursor.execute("TRUNCATE TABLE users CASCADE")
    conn.commit()
    yield
    cursor.execute("TRUNCATE TABLE users CASCADE")
    conn.commit()
```

Слайд 6: Независимость от времени

Проблема:

```
def test_user_age():
    user = create_user(birth_date="2000-01-01")
    age = calculate_age(user.birth_date)
    assert age == 24  # Ошибка в 2025!
```

Решение - мок времени:

```
from unittest.mock import patch
from datetime import datetime

def test_user_age():
    with patch('datetime.datetime') as mock_datetime:
        mock_datetime.now.return_value = datetime(2024, 1, 1)
        mock_datetime.side_effect = lambda *args, **kw: datetime(*args, **kw)

        user = create_user(birth_date="2000-01-01")
        age = calculate_age(user.birth_date)
        assert age == 24  # Всегда верно
```

Слайд 7: Request ID

Что такое Request ID?

Уникальный идентификатор запроса для трассировки.

Зачем нужен:

- Связать логи разных компонентов
- Найти все записи для одного запроса
- Диагностика проблем

Пример:

```python
import uuid

@app.before_request
def add_request_id():
    request.id = str(uuid.uuid4())
    g.request_id = request.id

@app.after_request
def add_request_id_header(response):
    response.headers['X-Request-ID'] = g.request_id
    return response
```

Слайд 8: Логирование с Request ID

Пример логирования:

```
import logging

logger = logging.getLogger(__name__)

@app.route('/api/users')
def get_users():
    request_id = g.request_id

    logger.info(f"[{request_id}] Getting users")

    try:
        users = get_users_from_db()
        logger.info(f"[{request_id}] Found {len(users)} users")
        return jsonify(users)
    except Exception as e:
        logger.error(f"[{request_id}] Error: {e}", exc_info=True)
        raise
```

Поиск в логах:

```
grep "abc123-def456" /var/log/app.log
```

Слайд 9: Воспроизведение ошибки

Шаги воспроизведения:

1. **Получить Request ID** из ошибки
2. **Найти в логах** все записи с этим ID
3. **Проанализировать** последовательность событий
4. **Создать тест** воспроизводящий проблему
5. **Исправить** ошибку
6. **Добавить регрессионный тест**

Пример:

```
# Логи
[abc123] Request received: POST /users
[abc123] Validation passed
[abc123] Database error: duplicate key value violates unique constraint
[abc123] Response: 409 Conflict
```

Слайд 10: Проверка результата операции

Не только HTTP статус!

Плохо:

```
def test_create_user():
    response = client.post('/users', json={'name': 'John'})
    assert response.status_code == 201  # ✓
```

Хорошо:

```
def test_create_user():
    response = client.post('/users', json={'name': 'John'})
    assert response.status_code == 201

    # Проверка в БД
    user = db.session.query(User).filter_by(name='John').first()
    assert user is not None
    assert user.email == 'john@example.com'

    # Проверка побочных эффектов
    assert len(user.orders) == 0
```

Слайд 11: Регрессионный тест

Что такое регрессионный тест?

Тест, который проверяет, что исправленная ошибка не вернётся.

Пример:

```
# Баг: email не проверялся на уникальность
def test_duplicate_email_regression():
    """Regression test for duplicate email bug"""
    # Arrange
    create_user("John", "john@example.com")

    # Act & Assert
    with pytest.raises(ValueError, match="Email already exists"):
        create_user("Jane", "john@example.com")
```

Когда добавлять:

- После исправления бага
- Перед релизом
- При изменении критического кода

Слайд 12: Интеграционный тест с реальной БД

Полный пример:

```
@pytest.fixture
def test_app_with_db():
    """Setup Flask app with test database"""
    app.config['SQLALCHEMY_DATABASE_URI'] = TEST_DATABASE_URL
    app.config['TESTING'] = True

    with app.app_context():
        db.create_all()
        yield app
        db.drop_all()

def test_create_user_integration(test_app_with_db):
    """Integration test with real database"""
    client = test_app_with_db.test_client()

    response = client.post('/users', json={
        'name': 'John',
        'email': 'john@example.com',
        'password': 'password123'
    })

    assert response.status_code == 201

    # Verify in database
    with test_app_with_db.app_context():
        user = User.query.filter_by(email='john@example.com').first()
        assert user is not None
        assert user.name == 'John'
```

Слайд 13: Тестирование внешних API

Мокирование внешних API:

```python
@pytest.fixture
def mock_external_api():
    """Mock external API responses"""
    with patch('requests.get') as mock_get:
        mock_get.return_value.json.return_value = {
            'status': 'ok',
            'data': {'user_id': 123}
        }
        yield mock_get

def test_sync_with_external_api(mock_external_api):
    """Test integration with external API"""
    result = sync_user_with_external_api(user_id=123)

    assert result['status'] == 'ok'
    mock_external_api.assert_called_once()
```

Почему мокать:

- Ненадёжные внешние сервисы
- Медленные запросы
- Стоимость API вызовов
- Нестабильные данные

Слайд 14: Логирование ошибок

Структурированное логирование:

```
import logging
import json

logger = logging.getLogger(__name__)

def log_error(request_id, error, context):
    """Structured error logging"""
    logger.error(
        json.dumps({
            'request_id': request_id,
            'error': str(error),
            'error_type': type(error).__name__,
            'context': context,
            'timestamp': datetime.now().isoformat()
        })
    )
```

Пример:

```
try:
    user = create_user(data)
except ValidationError as e:
    log_error(
        request_id=g.request_id,
        error=e,
        context={'data': data, 'endpoint': '/users'}
    )
```

Слайд 15: Мониторинг и алерты

Метрики для мониторинга:

- Количество ошибок по типу
- Время ответа (p50, p95, p99)
- Количество запросов
- Статус-коды (4xx, 5xx)
- Размер очереди

Пример с Prometheus:

```
from prometheus_client import Counter, Histogram

error_counter = Counter('app_errors_total', 'Total errors', ['error_type'])
response_time = Histogram('app_response_time_seconds', 'Response time')

@app.route('/api/users')
@response_time.time()
def get_users():
    try:
        users = get_users()
        return jsonify(users)
    except Exception as e:
        error_counter.labels(error_type=type(e).__name__).inc()
        raise
```

Слайд 16: Практическое задание

Задание 1: Интеграционный тест с БД

- Создать тест с реальной БД
- Подготовить тестовые данные
- Проверить результат в БД

Задание 2: Request ID

- Добавить Request ID в приложение
- Логировать с Request ID
- Найти запрос в логах

Задание 3: Регрессионный тест

- Создать тест для известного бага
- Исправить баг
- Убедиться что тест проходит

Задание 4: Мониторинг

- Добавить метрики
- Настроить логирование
- Создать алерт

Слайд 17: Вопрос для собеседования

Вопрос: Составьте последовательность диагностики и объясните, какой тест воспроизводит найденную проблему.

Ожидаемый ответ:

1. Получить Request ID из ошибки
2. Найти все логи с этим Request ID
3. Проанализировать последовательность событий
4. Определить место ошибки (БД, API, бизнес-логика)
5. Создать интеграционный тест воспроизводящий проблему
6. Исправить ошибку
7. Добавить регрессионный тест
8. Проверить что тест проходит